Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación

Autómatas y Lenguajes Formales

ER-AFD

Convertidor de expresiones regulares a autómatas finitos deterministas

CASOS DE PRUEBA

Catedrático: René Ornélyz

Estudiante: Josue Garcia

Guatemala, octubre de 2026

1. Entorno de ejecución

Elemento	Valor
Sistema operativo	Ubuntu 26.04.1 LTS, 64 bits (usuario regular, sin root)
Python	3.14.4
Make	GNU Make 4.4.1
Instalación	`git clone`, `make config` y `make install` (ejecutable `er-afd` en `~/.local/bin`)

Las capturas de pantalla se tomaron directamente de la terminal durante la ejecución. Después de cada comando se muestra `echo $?` con el código de retorno: **1** = cadena aceptada, **0** = cadena rechazada y **2** = error en la expresión regular.

Resumen

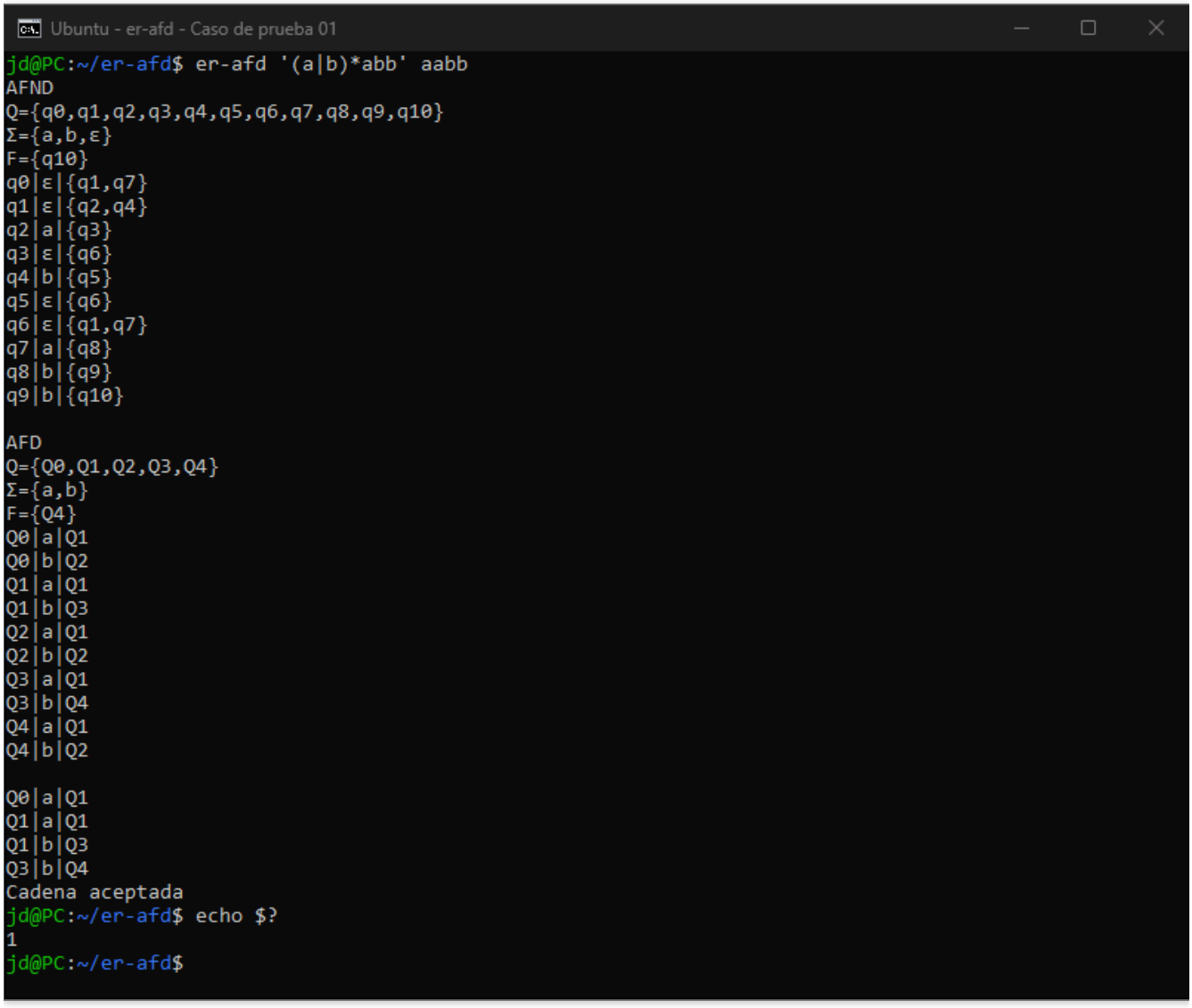
#	Objetivo	ER	Cadena	Esperado	Obtenido
1	Ejemplo clásico, tablas completas	`(a|b)*abb`	`aabb`	1	1 ✓
2	Rechazo en estado no final	`(a|b)*abb`	`abab`	0	0 ✓
3	ε explícito y su posición en Σ	`aε|b`	`a`	1	1 ✓
4	Comodín `.`	`a.b`	`a#b`	1	1 ✓
5	Conjunto con rango	`[a-c]x*`	`bxx`	1	1 ✓
6	Conjunto negado	`[^abc]\N`	`z7` / `a7`	1 / 0	1 / 0 ✓
7	Secuencias de escape	`a\*b\\`	`a*b\`	1	1 ✓
8	Código hexadecimal `\h`	`\h41\h20\N`	`A 5`	1	1 ✓
9	Espacios con escape y comillas	`hola\ mundo`	`"hola mundo"`	1 / 1	1 / 1 ✓
10	Clases `\A`, `\a`, `\z`	`\A\a*\h20\z\z*`	`Ana Lopez`	1	1 ✓
11	Detección real: placa vehicular	`[PMCAOTU](-|ε)\N\N\N\A\A\A`	`P-123ABC` / `P-123abc`	1 / 0	1 / 0 ✓
12	Cadena vacía	`a*`	(vacía)	1	1 ✓
13	Errores de sintaxis	`(a|b`, `[z-a]`, `a\h4`	`a`	2	2 ✓
14	Opciones `-v` y `-m`	`(a|b)*abb`	`abb`	1	1 ✓
15	`make run`	`\N\N*(\.\N\N*|ε)`	`3.1416`	1	1 ✓

16	Pruebas automáticas	`make test`	358 pruebas	**358** ✓
17	Métricas de precisión	`make metrics`	≥ 95 %	**100 %** ✓
18	Compilación desde cero	`make clean && make build`	ejecutable	✓

2. Casos de conversión y evaluación

Caso 1. Ejemplo clásico `(a|b)*abb`: cadena aceptada

Objetivo: verificar la salida completa: AFND de Thompson, AFD por subconjuntos y recorrido. **Esperado:** AFND de 11 estados (q0–q10), AFD de 5 estados con F={Q4}, recorrido Q0→Q1→Q1→Q3→Q4 y retorno 1.

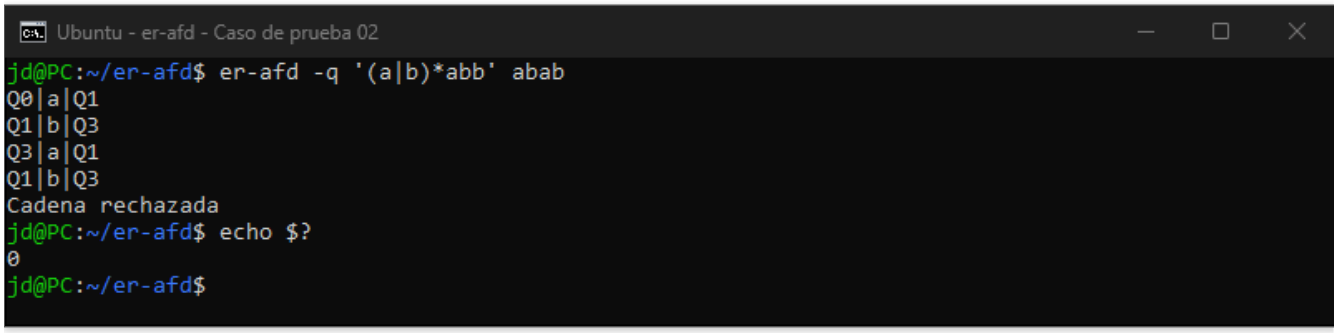


Captura 1. `er-afd '(a|b)*abb' aabb`

Resultado: correcto. Las tablas coinciden con la figura 3.34 y el ejemplo 3.21 de Aho et al.

Caso 2. Cadena rechazada al terminar en un estado no final

Objetivo: comprobar el rechazo cuando todas las transiciones existen pero el último estado no es de aceptación. Se usa `-q` para omitir las tablas, que son las del caso 1.

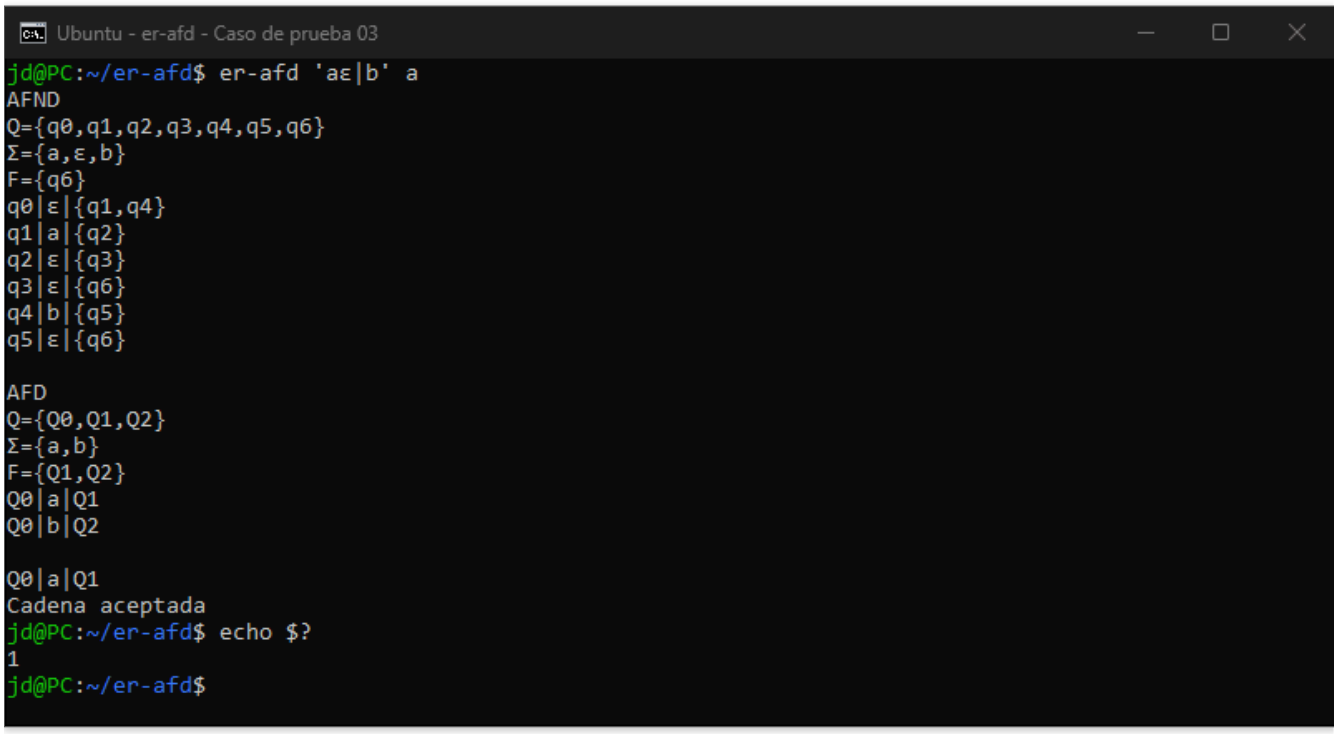


Captura 2. `er-afd -q '(a|b)*abb' abab`

Resultado: correcto. Termina en Q3, que no pertenece a F={Q4}, y devuelve 0.

Caso 3. Símbolo ε explícito

Objetivo: cuando ε aparece en la ER, ocupa en Σ del AFND la posición en que aparece (Σ={a,ε,b}) y no se incluye en Σ del AFD.

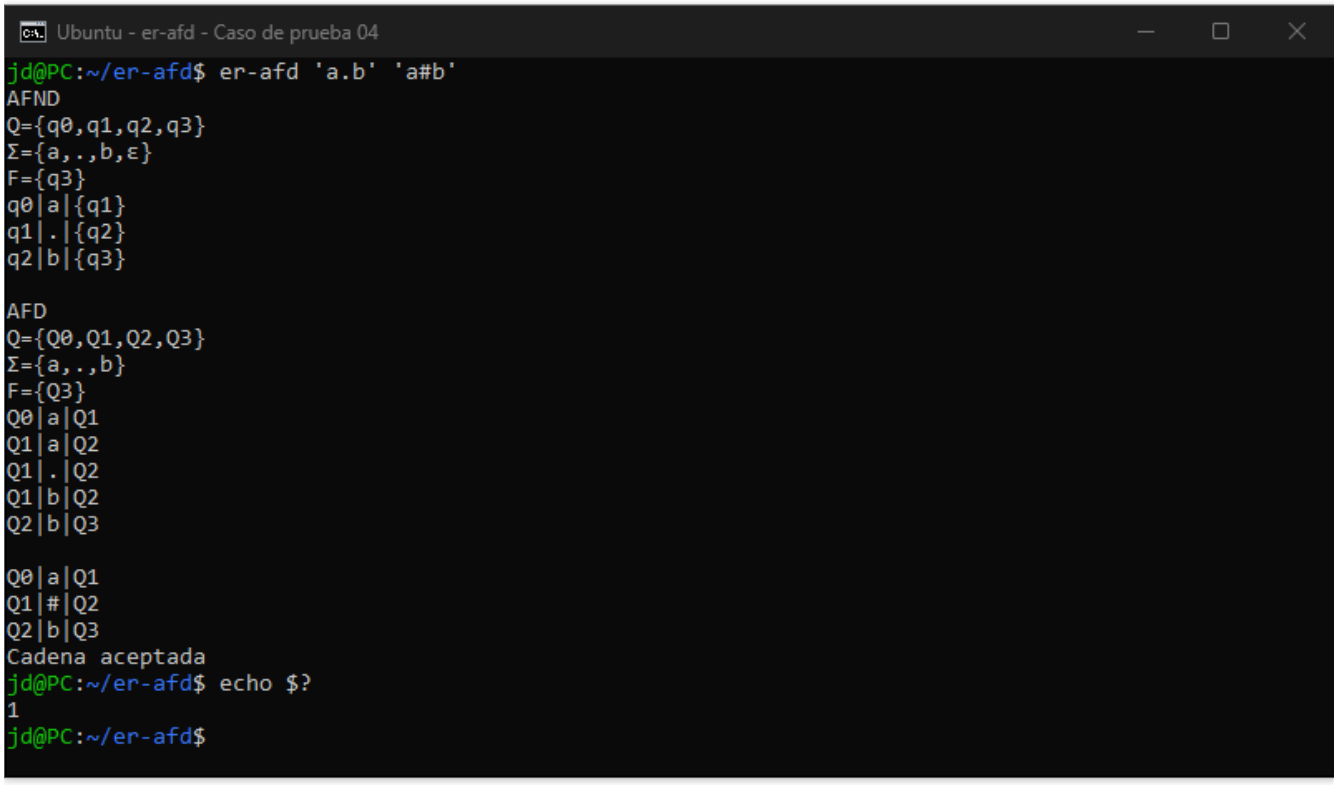


Captura 3. `er-afd 'aε|b' a`

Resultado: correcto.

Caso 4. Comodín `.`

Objetivo: `.` forma parte de Σ y acepta cualquier símbolo. En el AFD, desde Q1 las clases `a`, `.` (cualquier otro) y `b` llevan a Q2, así que el autómata sigue siendo determinista.

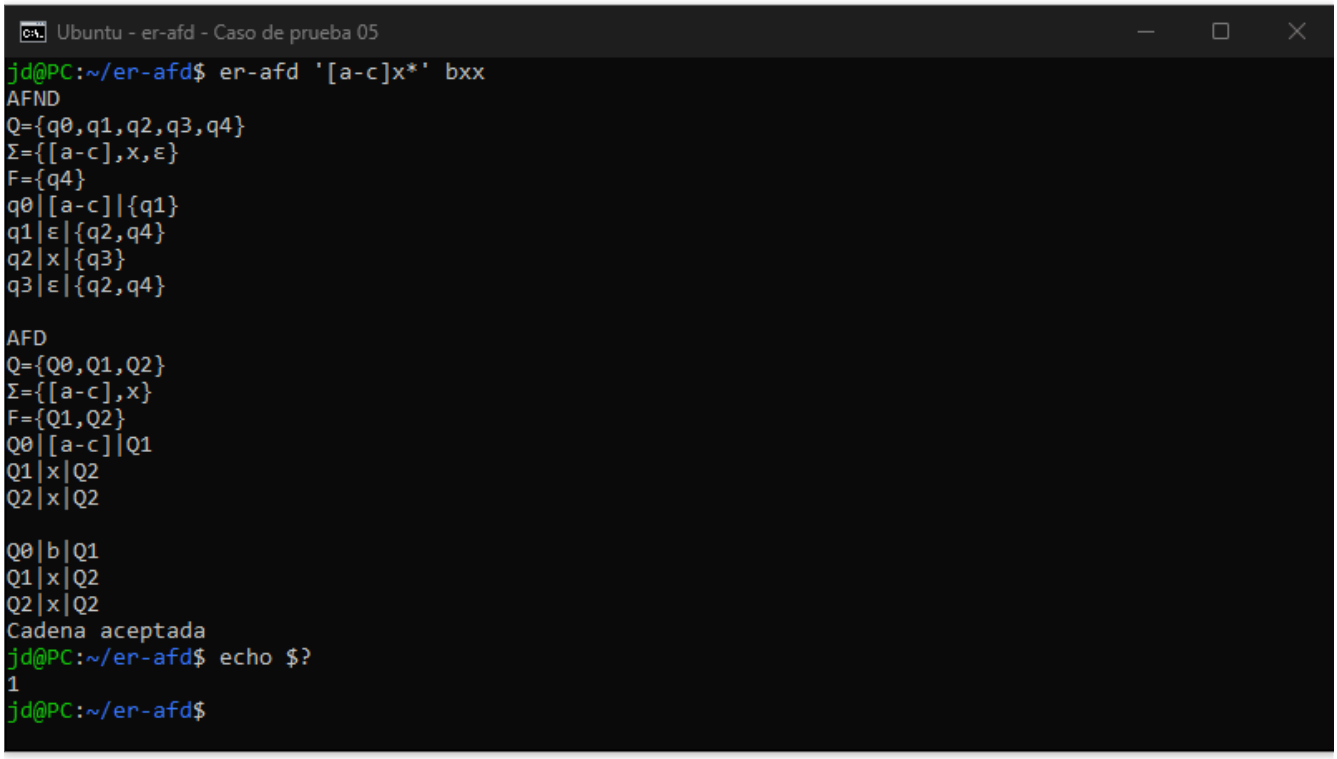


Captura 4. `er-afd 'a.b' 'a#b'`

Resultado: correcto. El carácter `#` se acepta mediante la transición de `.`.

Caso 5. Conjunto con rango `[a-c]`

Objetivo: un conjunto es un único símbolo de Σ; la cadena usa `b`, que pertenece al rango.

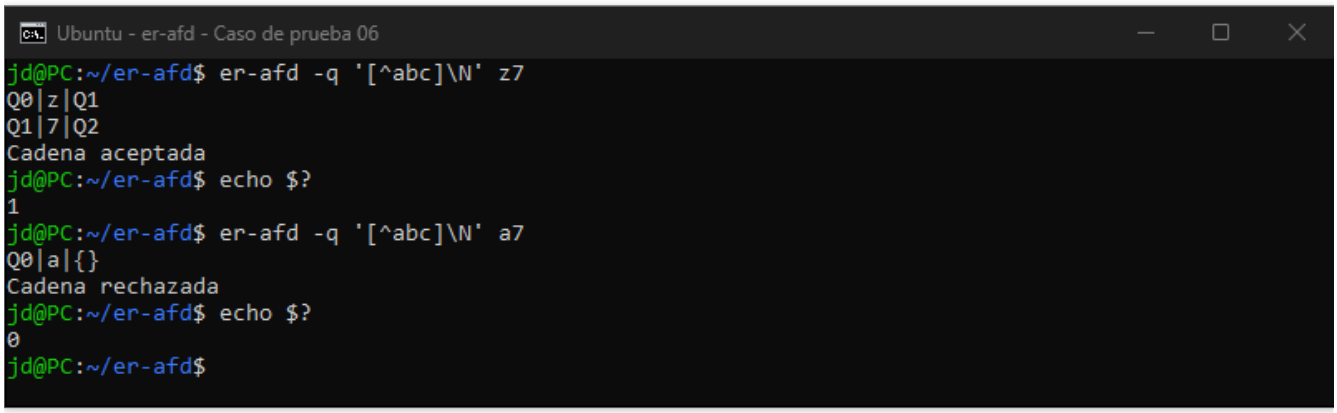


Captura 5. `er-afd '[a-c]x*' bxx`

Resultado: correcto.

Caso 6. Conjunto negado `[^abc]`

Objetivo: `z` no está en {a, b, c}, así que se acepta; `a` sí está, así que no tiene transición desde Q0. En ese caso el recorrido muestra `{}` (conjunto vacío) y la cadena se rechaza.

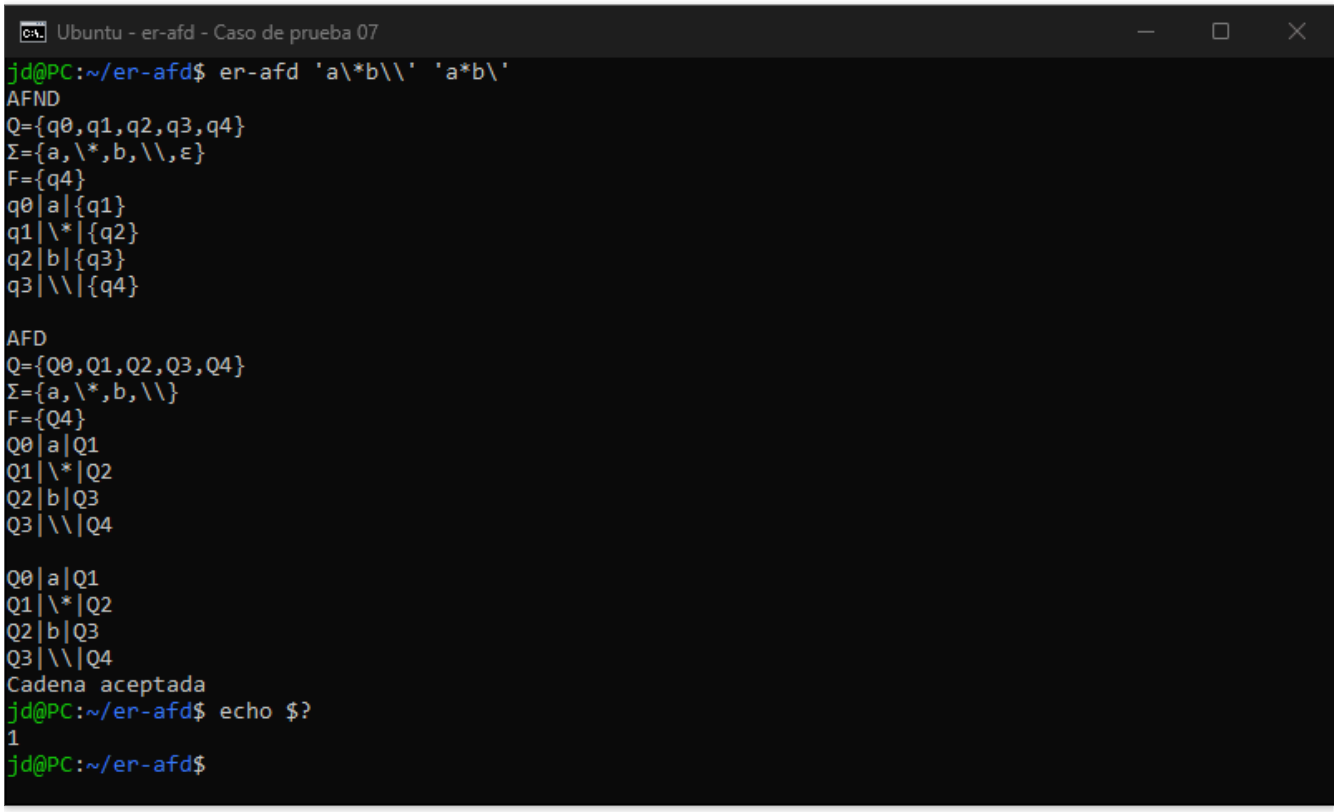


Captura 6. `er-afd -q '[^abc]\N' z7` y `a7`

Resultado: correcto.

Caso 7. Secuencias de escape

Objetivo: `\*` es el carácter asterisco (no la cerradura) y `\\` es la diagonal invertida. En las tablas se muestran escapados para no confundirlos con operadores.

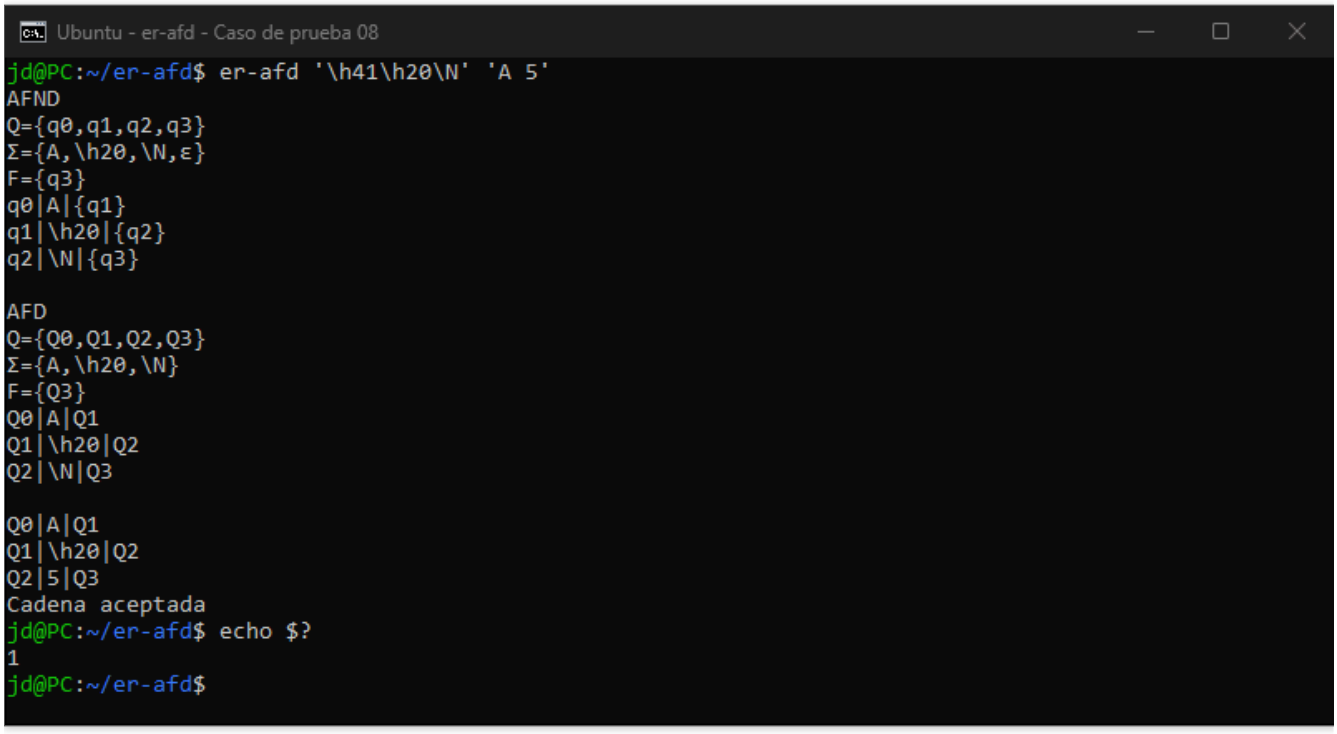


Captura 7. `er-afd 'a\*b\\' 'a*b\'`

Resultado: correcto.

Caso 8. Caracteres por código hexadecimal

Objetivo: `\h41` = `A` y `\h20` = espacio. El espacio se muestra como `\h20` en las tablas.

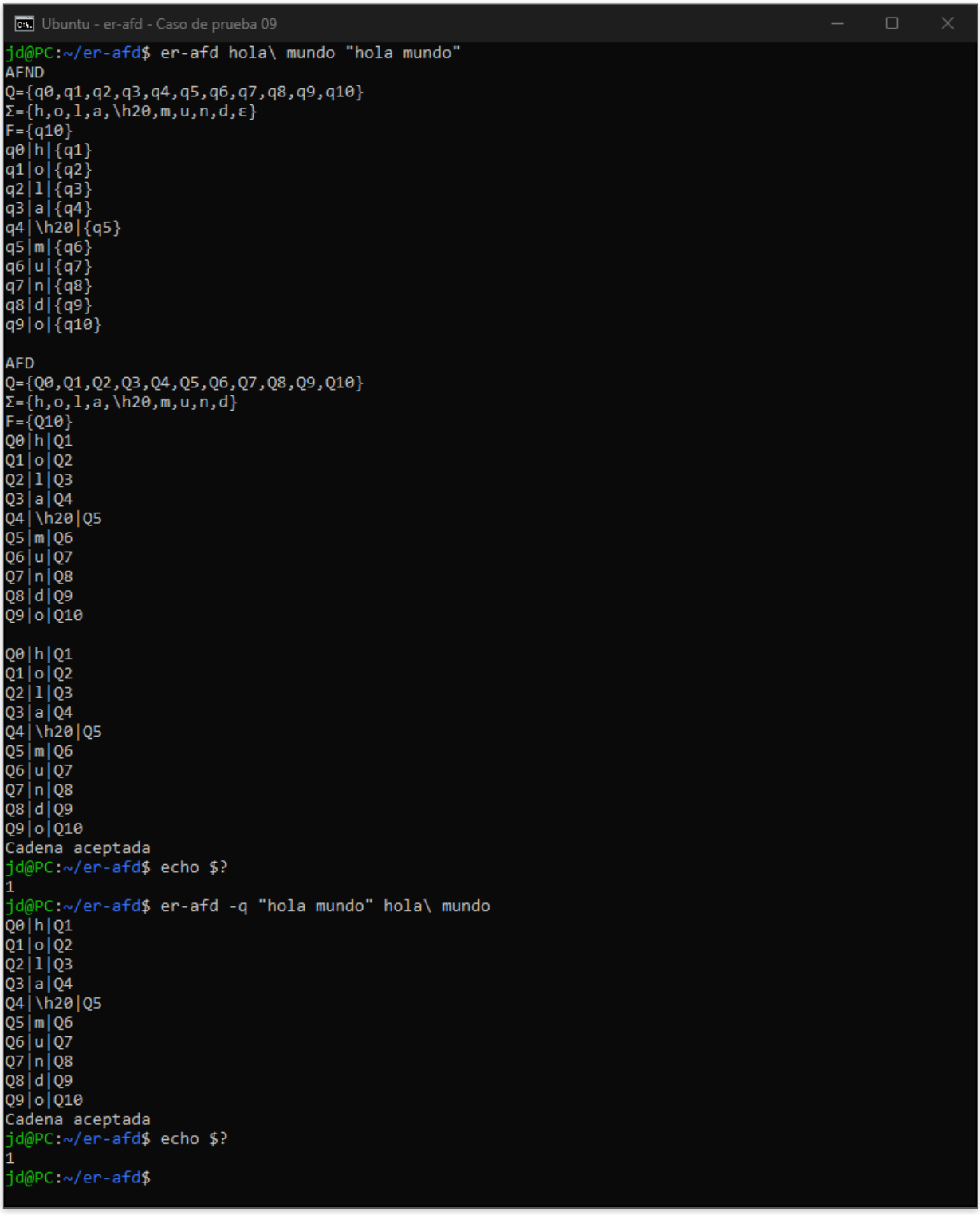


Captura 8. `er-afd '\h41\h20\N' 'A 5'`

Resultado: correcto.

Caso 9. Espacios con escape y con comillas dobles

Objetivo: las dos formas que indica el enunciado para escribir espacios, `hola\ mundo` y `"hola mundo"`, funcionan tanto en la ER como en la cadena.

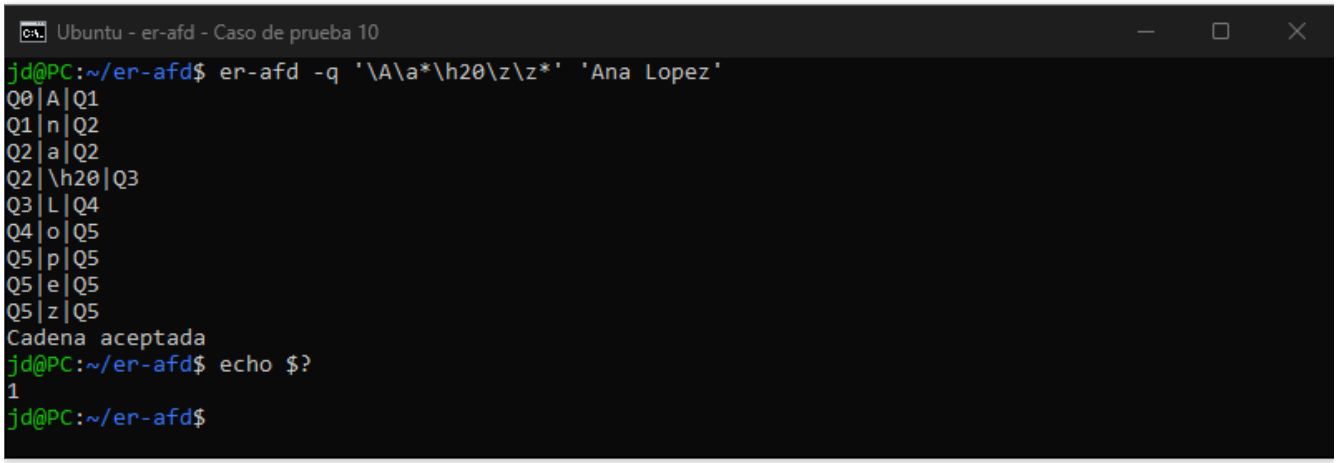


Captura 9. Espacios en la ER y en la cadena.

Resultado: correcto.

Caso 10. Clases `\A`, `\a` y `\z`

Objetivo: nombre y apellido: mayúscula inicial, minúsculas, un espacio y luego letras.

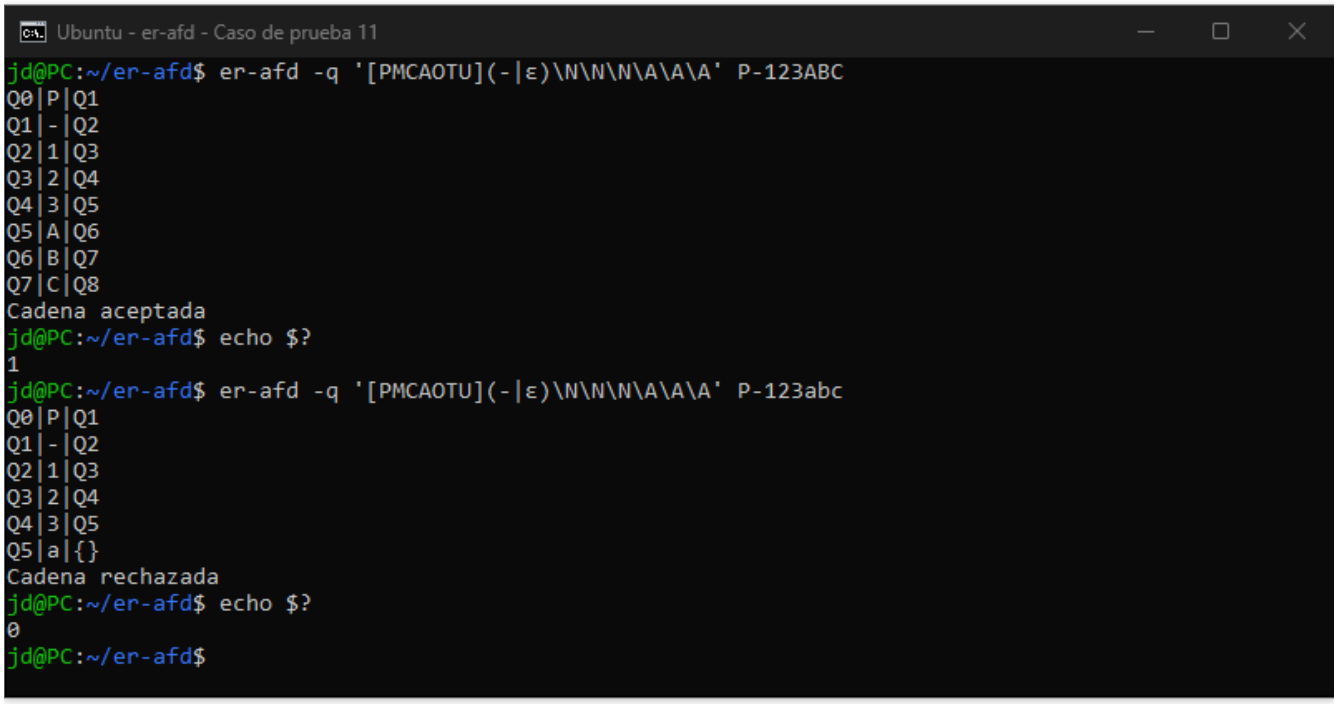


Captura 10. `er-afd -q '\A\a*\h20\z\z*' 'Ana Lopez'`

Resultado: correcto.

Caso 11. Detección real: placa vehicular de Guatemala

Objetivo: tipo de vehículo, guion opcional, tres dígitos y tres letras mayúsculas. La segunda cadena tiene minúsculas y se rechaza en el sexto carácter.

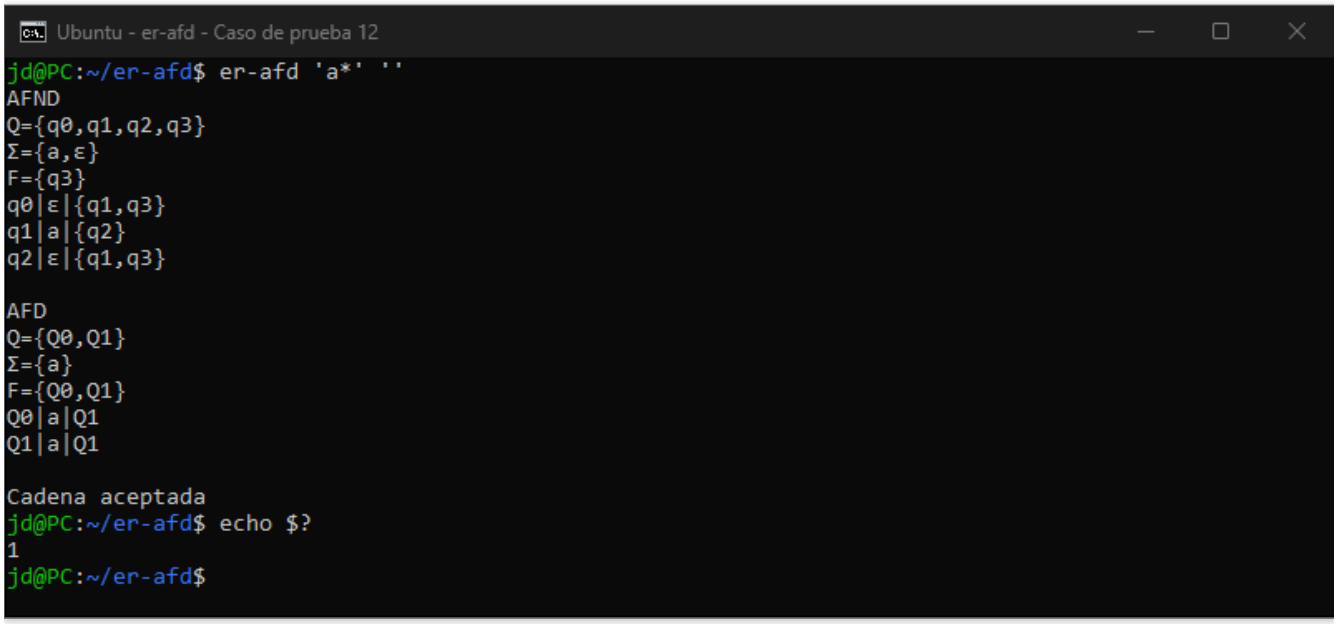


Captura 11. Placa válida e inválida.

Resultado: correcto.

Caso 12. Cadena vacía

Objetivo: la cadena vacía se acepta si Q0 es de aceptación; en ese caso no hay pasos que mostrar.

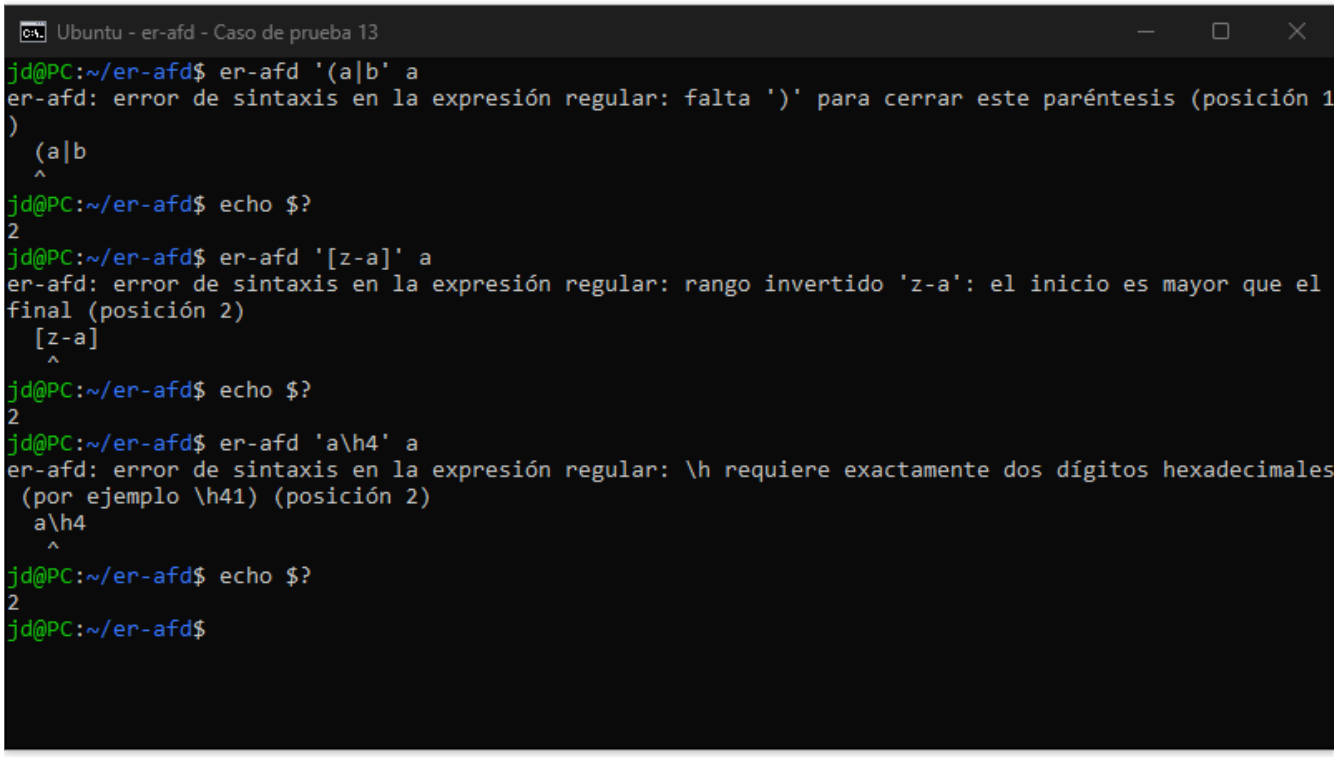


Captura 12. `er-afd 'a*' ''`

Resultado: correcto.

Caso 13. Errores de sintaxis

Objetivo: una ER mal formada produce un mensaje que explica el error, señala la posición y devuelve 2.

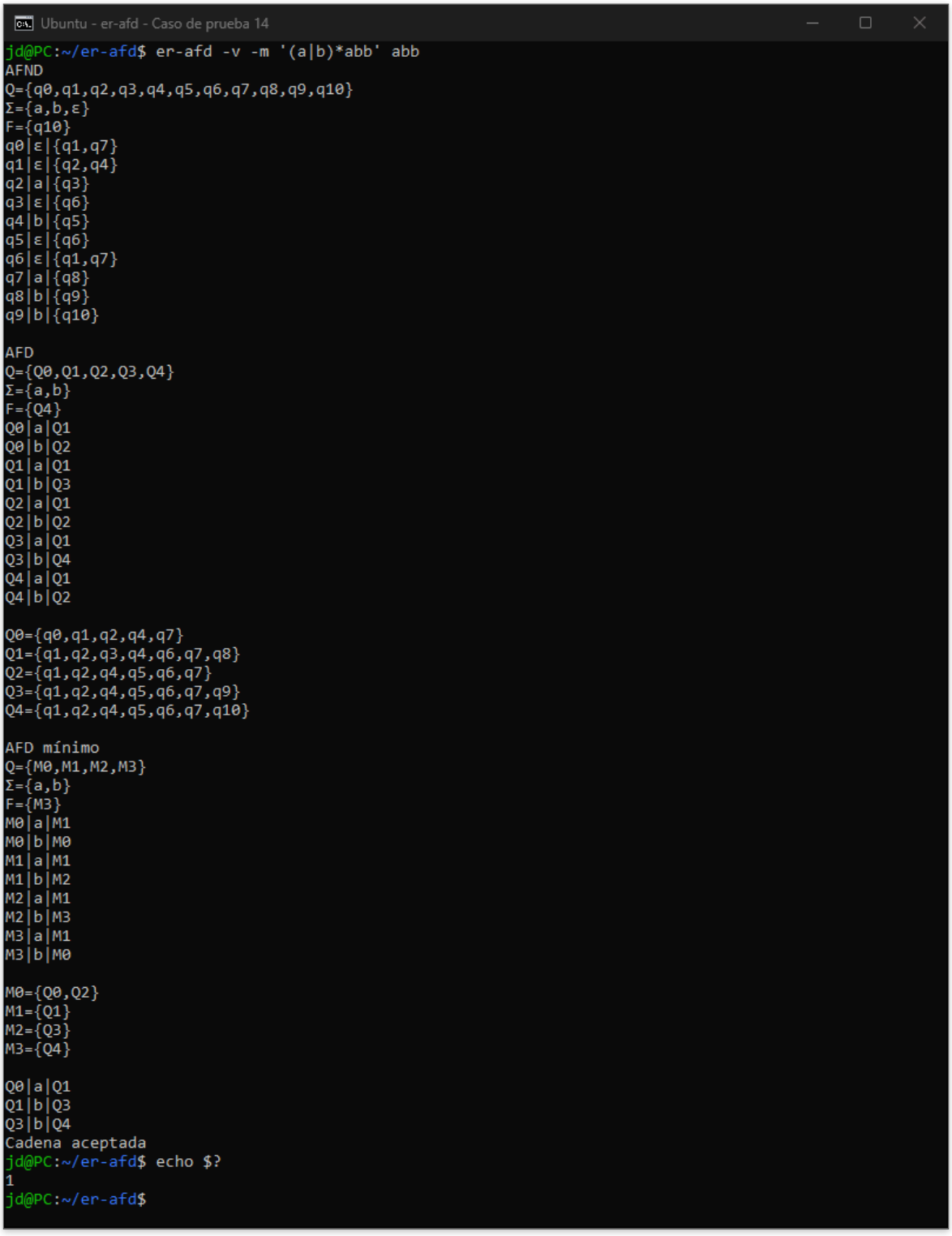


Captura 13. Paréntesis sin cerrar, rango invertido y `\h` incompleto.

Resultado: correcto.

Caso 14. Opciones adicionales `-v` y `-m`

Objetivo: `-v` muestra el subconjunto del AFND que representa cada estado del AFD; `-m` agrega el AFD mínimo, donde M0 = {Q0, Q2}.



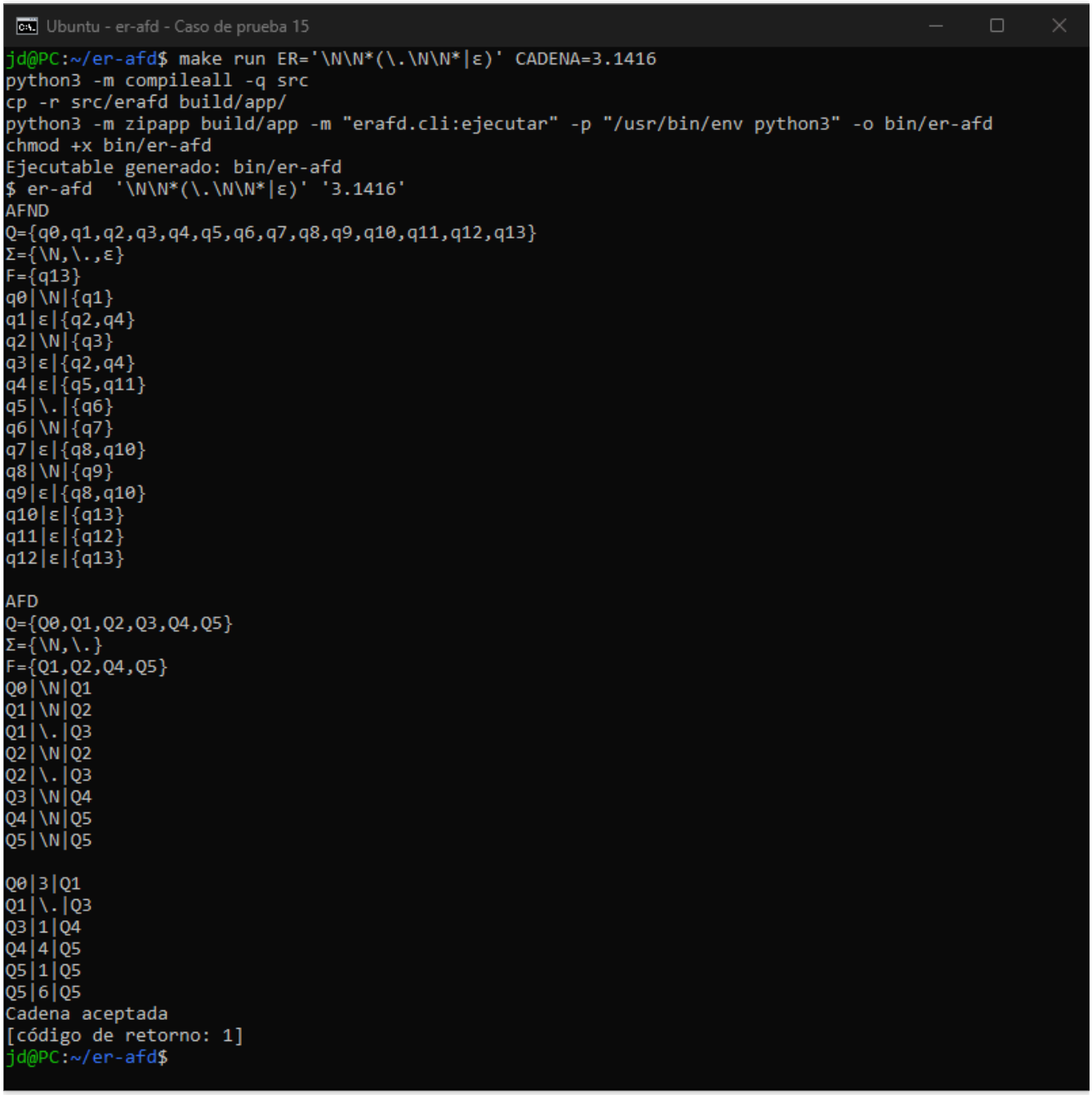
Captura 14. `er-afd -v -m '(a|b)*abb' abb`

Resultado: correcto.

3. Compilación y pruebas automáticas

Caso 15. `make run`

Objetivo: ejecutar el programa con el Makefile, recibiendo la ER y la cadena como variables. Aunque el programa devuelve 1 al aceptar, `make` no lo reporta como error.

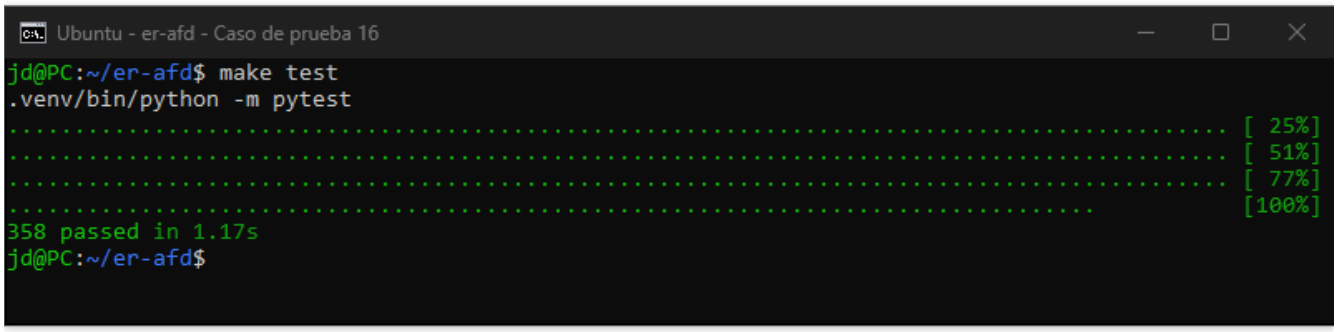


Captura 15. Número decimal con `make run`.

Resultado: correcto.

Caso 16. Pruebas automáticas

Objetivo: ejecutar las 358 pruebas (pytest). Cubren el analizador sintáctico, Thompson, los subconjuntos, la simulación, la interfaz de línea de comandos, el corpus de detecciones reales, la comparación con cadenas aleatorias y la verificación de que no se usan librerías de regex.

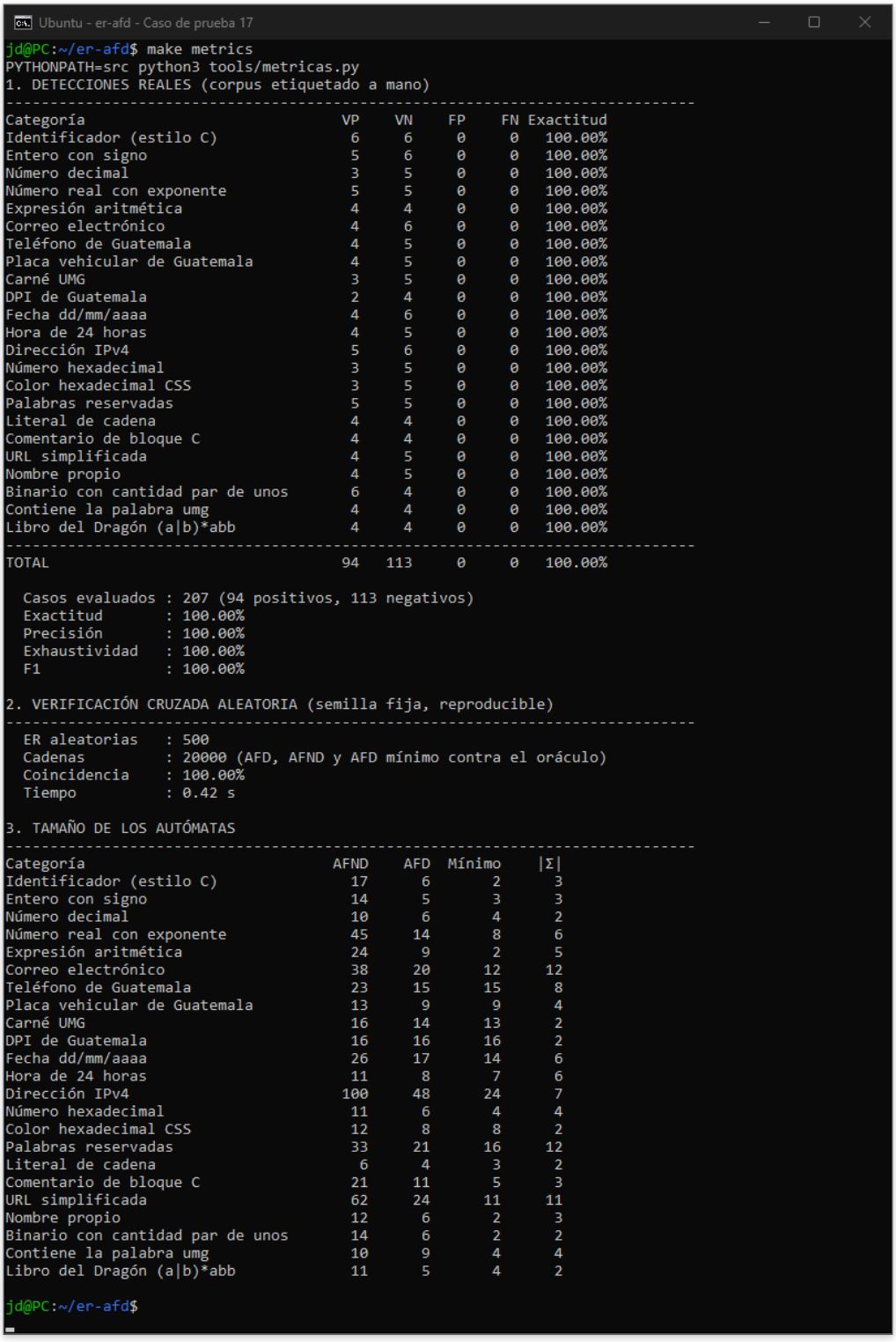


Captura 16. `make test`

Resultado: 358 pruebas aprobadas.

Caso 17. Métricas de precisión

Objetivo: medir la precisión de detección en 207 cadenas reales etiquetadas a mano (23 categorías) y en 20 000 cadenas aleatorias comparadas contra un oráculo independiente.



Captura 17. `make metrics`

Resultado: 100 % de exactitud, precisión, exhaustividad y F1, sin falsos positivos ni falsos negativos.

Caso 18. Compilación desde cero

Objetivo: `make clean` elimina lo generado y `make build` produce el ejecutable `bin/er-afd`.



Captura 18. `make clean && make build`

Resultado: correcto.